55

访问控制

本章作者：**Peter Reiher（UCLA）**

55.1 引言

现在，我们知道了自己的安全目标是什么，对想要实施的安全策略（security policy）至少有了大致的认识，也掌握了一些证据，知道是谁在请求那些可能违反（也可能不违反）这些策略的系统服务。接下来，我们需要把这些信息变成可以执行的东西，变成一段软件能替我们完成的工作。

这里有两个重要的步骤：

1. 判断这个请求是否符合我们的安全策略。
2. 如果符合，就执行这个操作；如果不符合，就确保它不会被执行。

第一步通常称为**访问控制**（access control）。我们要确定：哪些系统资源或服务，可以由哪些参与方，以哪些方式，在哪些情况下访问。归根结底，这又是一个非常契合计算机思维的二元决定：是或否。可是，这个决定该怎么做呢？为了让问题更具体，来看这样一个例子：用户 X 想读写文件 /var/foo。在底层，这多半意味着一个以用户 X 的身份运行的进程（process）发起了类似下面这样的系统调用（system call）：

open(”/var/foo”, O_RDWR)

注意，这里说的不是 Linux 的 `open()` 调用。Linux 的 `open()` 是一个具体实现，它用一种特定的方式处理访问控制。我们讨论的是一般意义上的问题：怎样控制对“打开文件”这类系统调用的访问。所以这里换了一种字体，提醒你注意这一点。

系统该怎样处理进程的这个请求，既要确保在安全策略禁止时不打开文件，又要确保在策略允许时打开文件？我们知道，系统调用会陷入（trap）操作系统，这让操作系统有机会做点什么来作出这个决定。从机制上讲，这个“什么”应该是什么呢？

关键问题：

如何判断是否应当批准一个访问请求

操作系统怎样决定，某个特定用户的某个特定进程在某个特定时刻发出的某个特定请求，是否应当批准？做这个决定要用到哪些信息？我们怎样设置这些信息，才能把想要在系统中实施的安全策略编码进去？

55.2 访问控制问题的几个重要方面

和往常一样，系统会运行某种算法来做这个决定。算法接受一些输入，产生一个二元输出，也就是“是否批准访问”的决定。在宏观层面，人们通常用**主体**（subject）、**客体**（object）和**访问**（access）来讨论访问控制。主体是想要执行访问的实体，可能是一个用户，也可能是一个进程。客体（object）是主体想要访问的东西，可能是一个文件，也可能是一个设备。访问是对客体（object）的某种特定操作方式，比如读或者写。所以，访问控制决定讨论的就是：某个特定的主体，是否获准以某种特定的访问方式访问某个特定的客体（object）。判断某个特定的主体是否获准对某个特定的客体（object）执行某种特定形式的访问，这个过程有时也称为**授权**（authorization）[1]。

一个相关的问题是：访问控制决定在什么时候做？系统每做一次这样的决定，都要把它所用的算法运行一遍。实现这个算法的代码称为 **reference monitor**。显然，我们有充分的理由确保它的实现既正确又高效。如果它不正确，就会作出错误的访问决定，这显然很糟糕。它的效率之所以重要，是因为每次用到它都会带来一些开销。为了尽量减少这些开销，我们也许不想每逢可能的时机都检查访问控制。可另一方面，还记得前面几章介绍过的完全仲裁（complete mediation）原则吗？那条原则说，每当有人请求某样东西时，我们都应当检查安全条件。

显然，我们需要在代价和安全收益之间取得平衡。不过，如果能找到一些有益的特殊情况，既能做到低开销，又不牺牲安全，那么至少在这些情况下，我们也许就不必在两者之间取舍了。

一种做法是让主体拥有只属于它们自己的客体（object）。如果某个客体（object）天然只属于某个主体（subject），而且这一点不可改变，系统就可以让这个主体（在操作系统中就是进程）自由地访问它。虚拟化（virtualization）让我们能创建这类虚拟客体（virtual object）。虚拟内存（virtual memory）就是一个绝佳的例子。进程可以自由地访问自己的虚拟内存[2]，在进程使用它的那一刻，操作系统不需要做专门的访问控制检查。这也是好事，否则每次进程引用内存时，我们都得运行一遍访问控制算法，系统会慢得离谱。对外围设备，我们也可以玩类似的虚拟化把戏。假设给一个进程分配了一个虚拟设备，它背后其实是操作系统控制的一个真实物理设备，而且其他进程都不许使用它，那么这个进程每次要用这个设备时，操作系统都不必检查访问控制。例如，可以根据一次初始的访问控制决定，把 GPU 的控制权交给某个进程；此后，这个进程就可以直接写 GPU 的内存、直接向它发指令，操作系统不再介入。

当然，前面讨论过，虚拟化在很大程度上是操作系统提供的一种假象。进程之间其实共享着内存、设备和其他计算资源。看起来只属于它们自己的东西，实际上是共享的，只是操作系统在幕后忙前忙后地维持着这个假象，有时还借助专门的硬件。这意味着，即使使用虚拟化资源的应用程序并不直接知情、也不直接参与，操作系统仍然必须确保只允许正当的访问方式。所以，单纯依靠虚拟化来保证访问正当，只是把问题推到了下一层：变成了保护操作

[1] 哇。你知道要让这么多个“particular”（特定的）在原文里上下排成一列有多难吗？这可是一整列 particular！不过，也许并不怎么（particularly）有趣。

[2] 几乎如此。还记得页表（page table）里那些决定某一页能否读、写或执行的位吗？不过，这里做运行时检查的不是操作系统，而是虚拟内存硬件。

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

系统的虚拟化功能。即使先把这个问题放在一边，我们迟早也得撇开这些廉价的特殊情况，去处理一般性的问题。主体 X 想读写客体 /tmp/foo。也许允许，也许不允许。现在怎么办?

针对这个问题，计算机科学家提出了两种基本方法，它们依赖不同的数据结构（data structure），用不同的方式作出决定。一种叫**访问控制列表**（access control list），另一种叫**能力**（capability）。其实，说是计算机科学家提出了这两种方法并不太准确，因为在计算机之外的场合，人们使用它们已经有几千年了。在讨论它们在操作系统中的实现之前，我们先从更一般的角度看看它们。

假设我们要开一家高级私人俱乐部（不妨叫它 Chez Andrea[3]），只接待最优秀的操作系统研究者和开发者。我们可不想让那些搞数据库或编程语言的人混进来，所以必须确保只有经过批准的客人才能进门。该怎么做呢? 一种办法是雇一个块头巨大、一脸凶相的门卫，他手里有一份所有获准会员的名单。有人想进俱乐部时，要先向门卫证明自己的身份，门卫再查这个人在不在名单上。如果来的是 Linus Torvalds 或 Barbara Liskov，门卫就放行；那些没能在操作系统领域出人头地的搞网络的平头百姓，就会被挡在门外。

另一种办法是在俱乐部门上装一把非常好的锁，再把这把锁的钥匙发给我们所有搞操作系统的朋友。如果 Jerome Saltzer 想进 Chez Andrea，他只要掏出钥匙开门就行了。要是有几个不懂操作系统的计算机体系结构专家想进来，他们没有钥匙，就只能待在外面。和前一种办法相比，我们省下了门卫的工资，不过得花钱买锁和钥匙[4]。等到操作系统领域又涌现出我们想接纳的新大牛，就得给他们配新钥匙。我们偶尔还可能犯错，把钥匙发给了不配拥有它的人；或者某个会员把钥匙弄丢了，这时就得确保那把钥匙再也打不开俱乐部的门。

同样的思路也可以用在计算机系统中。早期的计算机科学家把类似锁和钥匙的那种方法叫作**基于能力的系统**（capability-based system），而把基于门卫和准入名单的方法叫作**访问控制列表系统**（access control list system）。所以，能力就像钥匙，或者电影票，或者坐地铁用的代币；访问控制列表（access control list）嘛，就像列表。那么，这在操作系统中是怎么运作的呢? 如果用的是能力，当属于用户 X 的某个进程想读写文件 /tmp/foo 时，它会把针对这个文件的一个能力交给系统。（你可能会问，在这个语境下，能力到底是什么? 问得好！我们后面会讲到。）如果用的是访问控制列表（简称 **ACL**），系统会在与 /tmp/foo 关联的 ACL 上查找用户 X，只有用户在名单上时才允许访问。无论哪种方式，检查都可以在请求访问（在我们的例子里就是调用 `open()`）的那一刻进行。检查发生在陷入操作系统之后、真正允许访问之前；如果访问控制检查没有通过，就提前退出并返回错误码。

从宏观上看，这两种做法听起来也许没有太大差别；但只要开始考虑需要运行的算法，以及支撑这个算法所需的数据结构，你很快就会发现它们差别很大。下面逐一来看。

[3]作者 Arpaci-Dusseau 夫妇想声明一下：安全部分各章里的这些名字都是作者 Reiher 定的，他们并没有逼他在本章和其他例子里一直用他们的名字。下面请继续正常阅读……

[4]注意，无论是访问控制列表（access control list）还是能力，我们都假定已经对想进俱乐部的人做过身份认证（authentication）。如果有个无名小卒戴着 Linus Torvalds 或 Barbara Liskov 的面具骗过了门卫，或者我们没有仔细确认对方真的是 Jerome Saltzer 就把钥匙交给了随便一个人，那就挡不住那些乌合之众了。抛开这个可爱的比喻，同样的问题在真实的计算机系统中也完全存在，这正是上一章详细讨论身份认证的原因。

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

55.3 用 ACL 实现访问控制

按照英国老式俱乐部的传统，如果 Chez Andrea 除了让会员使用图书室、餐厅、台球室和其他公共空间之外，还给每位会员一间私人房间，会怎么样？这时，我们不仅要确保只有会员才能进俱乐部，还要确保 Ken Thompson（大家都知道他有点调皮 [T84]）不能溜进 Whitfield Diffie 的房间，在他床上搞恶作剧。我们可以用一张大的访问控制列表（access control list），规定每个房间允许谁怎样进入，但那样会变得难以管理。何不给俱乐部的每个房间各配一张 ACL 呢？

在依靠 ACL 做访问控制的典型操作系统中，我们对文件也是这样处理的。每个文件都有自己的访问控制列表（access control list），这样列表更简单、更短，访问控制检查也更快。所以，在 ACL 系统中，我们的 `open()` 调用会检查 /tmp/foo 的列表，而不是一张编码了系统中每个文件所有访问权限的 ACL。

这个 `open()` 调用陷入操作系统时，操作系统会查看当前运行进程的 PCB，确定这个进程的属主（owner）是谁。这个数据结构表明，进程属于用户 X。接下来，系统必须拿到 /tmp/foo 的访问控制列表（access control list）。ACL 也是一种文件元数据（metadata），类似于我们在“文件和目录”一章中讨论过的那些东西。所以，它很可能和这个文件的其他元数据存放在一起，或者存放在它们附近。总之，我们要从持久存储（persistent storage）中取出这张列表，然后在列表中查找 X。X 要么在，要么不在。如果不在，X 就没有访问权。如果在，我们通常还要更进一步，判断 X 的 ACL 表项是否允许所请求的访问类型。在我们的例子中，X 想以读写方式打开 /tmp/foo。也许 ACL 允许 X 以读方式打开这个文件，但不允许以写方式打开。这时，系统就会拒绝这次访问，并给进程返回一个错误。

原理上，这并不复杂。可还记得“魔鬼藏在细节里”这句话吗？魔鬼还在。我们来考虑其中一些细节。例如，ACL 到底持久地存放在哪里？对大多数资源来说，它确实需要持久保存，因为 ACL 实际上编码了我们选定的安全策略，而这种策略多半不会经常变动。所以它存放在闪存盘（flash drive）或磁盘上的某个地方。除非做了缓存（cached），否则每次有人试图打开文件，我们都得从设备上把它读出来。正如持久性（persistence）部分讨论过的，在大多数文件系统（file system）中，要从文件里真正读到任何信息，本来就得读好几次设备。难道还要再读一次，才能拿到这个文件的 ACL？如果是这样，ACL 应该放在设备上的什么位置，才能保证访问起来快？最好它靠近我们本来就要读的某样东西，甚至就是那样东西的一部分。这样看，有几个可能的位置：文件的目录项（directory entry）、文件的 inode，或者文件的第一个数据块（data block）。至少，我们希望 ACL 靠近其中某个位置；如果能直接放在其中某一处（比如 inode）里，可能更好。

这又引出另一个恼人的细节：这张列表有多大？如果按最直观的办法，列出实际的用户 ID 和访问方式，那么原则上这张列表可以任意大，最多可以达到系统所知的用户数。对某些系统来说，这可能是几千个表项。但通常，一个文件只属于一个用户，往往也只有这个用户，或许再加上几个朋友，才能使用它。所以，我们不会想在每张 ACL 里都预留足够的空间，把所有可能的用户都列进去，因为大多数用户不会出现在大多数 ACL 中。当然也有例外：很多文件应当允许所有用户以某种方式（比如读或执行）使用。毕竟，常用的可执行程序（比如 `ls` 和 `mv`）都存放在文件中，我们也会像对待其他文件一样，对它们实施访问控

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

制。用户们会共用同样的字体文件、网络配置文件，等等。我们必须允许所有用户访问这些文件，否则他们在系统上几乎什么也做不了。

所以，最直观的实现会给每个文件预留一张大列表，有些文件的列表被完全填满，另一些则几乎是空的。这显然很浪费。对于填满的列表，还有一个令人担忧的细节：每次想在列表中检查访问权限，都得搜索它。现代计算机搜索一个有一千个表项的列表相当快，但如果我们一直要做这样的搜索，就会给系统增加大量不必要的开销。我们可以用变长的访问控制列表（access control list）来解决这个问题，只为每张列表分配它实际需要的空间。花点时间想一想，你会怎样把这类元数据塞进我们学过的那几种文件系统，又会对性能有什么影响。

幸运的是，在大多数情况下，我们可以受益于最初的贝尔实验室 Unix 系统留下来的一点遗产。在那个远古时代，计算机科学的巨人们还在地球上（至少是在新泽西州的某些地方）漫游，持久存储稀缺而昂贵，他们根本负担不起为每个文件存一张大 ACL。实际上，他们算了一下，发现每个文件的 ACL 大约只负担得起九位。九位实在不多，好在那些早期的 Unix 设计者聪明绝顶，足以弥补硬件的不足。他们琢磨了自己的问题，发现真正关心的访问方式实际上只有三种（对大多数文件来说是读、写和执行），而且每张访问控制列表（access control list）只要三个表项，就能应付大多数安全策略。当然，如果每个表项的每种访问方式都用一位，九位就已经用光了，再没有位能用来指明这个表项是针对谁的。于是，他们巧妙地把访问控制列表（access control list）上的表项分成三组。一组是文件的属主，属主的身份本来就已经存在 inode 里了。一组是某个特定用户组的成员，组 ID 也存在 inode 里。最后一组是其他所有人，也就是既不是属主、也不是属主所在组成员的所有人。这一组不需要用任何位来存储，因为它就是属主和组的补集。

这个方案不仅解决了 ACL 占用存储空间的问题，还解决了访问和检查 ACL 的开销问题。不管对文件做什么，你几乎都要先访问它的 inode；所以，如果 ACL 嵌在 inode 里，就不需要额外的寻道（seek）和读取来获取它。而且，不必在任意长度的列表里搜索，只要对几个位做一点简单的逻辑运算，就能回答访问控制的问题。直到今天，大多数采用符合 Posix 标准的文件系统的系统，仍然靠这套逻辑来回答这个问题。当然，这种方法也有局限，因为它无法表达复杂的访问方式和共享关系。正因为如此，一些现代系统（比如 Windows）提供了扩展，允许使用更通用的 ACL；但很多系统依然依赖久经考验的 Unix 风格九位 ACL[5]。

ACL 有一些好的特点，也有一些局限。先说优点。第一，如果你想弄清楚谁可以访问某个资源，用 ACL 时这个问题很好回答，直接看 ACL 本身就行。第二，如果你想改变能访问某个客体（object）的主体集合，只需修改 ACL，因为再没有别的东西能给用户访问权。第三，由于 ACL 通常和文件本身放在一起，或者放在它附近，只要你能拿到文件，就能拿到所有相关的访问控制信息。这一点在分布式系统（distributed system）中尤其重要；不过只要设计时让 ACL 靠近文件或它的 inode，它对所有系统的性能也都有好处。

再说不太理想的地方。第一，ACL 要求你解决前面提到的那些问题：要把访问控制信息存放在文件附近的某个地方，还要应付在长列表中搜索可能带来的高开销。我们介绍了一些在大多数系统中效果不错的实用方案，但这些方案也限制了 ACL 能做的事。第二，如果你想

[5]真实的历史要更复杂一些。CTSS 系统提供了一种比 Unix 更受限的压缩 ACL [C+63]，而 Multics 系统在一种更通用的访问控制列表（access control list）中引入了组的概念，这种列表由用户名和组名的字符串组成 [S74]。所以，Unix 的方法是这些更早系统的杂交产物。

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

补充：命名空间

我们刚刚遇到了分布式系统中一个有趣而又困难的问题：名字在不同的机器上意味着什么？在单台计算机上，这个**命名空间**（name space）问题相对容易。如果为一个新东西选的名字已经被占用了，就不允许分配它。所以，在这个系统上，无论是哪个用户或进程使用某个名字，它指的都是同一样东西。对你和你计算机上的所有其他用户来说，/etc/password 都是同一个文件。

可是，由多台计算机组成的分布式系统呢？如果你想同样保证名字唯一、而且所有人都理解一致，就得确保 UCLA 某台机器上的人不会创建一个威斯康星大学已经在用的名字。怎么做到呢？不同的答案各有利弊。一种办法是不去管它，接受各个命名空间互不相同这一事实，我们对进程 ID（process ID）就是这么做的。另一种办法是要求由一个权威机构来审批名字的选用，AFS 处理文件名的创建大致就是这样。还有一种办法是把命名空间分成若干部分，交给各个参与者，允许它们在自己那部分里随意取名，但不能取其他名字，万维网和 IPv4 地址空间（address space）就是这样处理这个问题的。这些答案没有哪个是绝对正确或绝对错误的。根据自己的需要设计命名空间，但要明白其中的含义。

弄清楚某个主体获准访问的全部资源，该怎么办？你得把系统中的每一张 ACL 都查一遍，因为这个主体可能出现在任何一张 ACL 上。第三，在分布式环境中，要让 ACL 生效，所有机器对身份必须有共同的认识。如果 cs.ucla.edu 上的某个用户想访问存放在 cs.wisconsin.edu 上的文件，威斯康星的机器就要拿 UCLA 提供的某个身份，去和存放在威斯康星的访问控制列表（access control list）比对。UCLA 的用户 remzi 和威斯康星的用户 remzi 真的是同一个主体吗？如果不是，你就可能让一个远程用户访问了他不该访问的东西。但要在多个不同的计算域之间维护一个一致的用户命名空间（name space），是很有挑战性的。

55.4 用能力实现访问控制

在计算机系统中，访问控制列表（access control list）并不是控制访问的唯一选择。几乎是，但不完全是。你还可以使用能力，也就是更像钥匙或门票的那种做法。Chez Andrea 可以给会员发钥匙，让他们凭钥匙入场。不同的房间可以配不同的钥匙，防止那些比较调皮的会员在别人房间里留下点小“惊喜”。每位会员都随身带着一串钥匙，凭这些钥匙进入俱乐部中自己有权进入的区域。和 ACL 一样，能力在计算机系统中也有很长的使用历史，Dennis 和 van Horn [DV64] 也许是最早的例子。Wulf 等人 [W+74] 介绍了 Hydra 操作系统，它把能力用作一种基本的控制机制。Levy [L84] 用一整本书总结了能力在早期硬件和软件系统中的应用。在能力系统中，运行着的进程拥有一组能力，这些能力规定了它的访问权限。如果你用的是纯粹的能力系统，任何地方都没有 ACL，这组能力就是这个进程访问权限的全部编码。Linux 和 Windows 不是这样工作的，但其他一些操作系统（比如 Hydra）研究过用这种方法处理访问控制。

在这种纯粹的能力系统中，我们该怎样执行那个 `open()` 调用呢？发起调用时，要么由你的应用程序把一个允许你的进程打开该文件的能力作为参数传进去，要么由操作系统替

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

你找到这个能力。无论哪种情况，操作系统都会检查这个能力是否允许你以读写方式打开文件 /tmp/foo。如果允许，操作系统就替你打开它。如果不允许，你的进程就会收到一个错误，数落它竟想打开一个自己没有相应能力的文件。（记住，我们这里说的不是 Linux。Linux 是用 ACL 而不是能力来判断是否允许一次 `open()` 调用的。）

这里有几个显而易见的问题。能力究竟是什么？显然我们说的不是金属钥匙或纸质门票。还有，操作系统怎样检查能力是否有效？能力最初又是从哪里来的？和计算机中的其他所有信息一样，能力就是一串位，是数据。考虑到要保护的资源可能很多，而能力必须针对某个具体资源，能力很可能相当长，也许还相当复杂。但归根结底，它们只是位。任何由一串位组成的东西，都有一些我们必须牢记的特性。例如，任何人都可以随意造出任何一串位，没有哪种位模式是专属的、或者保留下来不让进程创建的。另外，如果一个进程有一份某组位的副本，再造出更多副本是轻而易举的事。第一个特性意味着，任何人都有可能造出自己想要的任何能力。第二个特性意味着，一旦某人有了一个有效的能力，就可以随意复制任意多份，还可能把它们存放在任何地方，包括另一台完全不同的机器上。

从安全的角度看，这听起来不太妙。如果一个进程需要一个具有特定位模式的能力，才能以读写方式打开 /tmp/foo，那它也许只要自己生成这个位模式，就能给自己弄到想要的文件访问权。这可不是我们对访问控制机制（access control mechanism）的期望。我们希望能力是不可伪造的。即使能解决这个问题，能力可以复制这一点也意味着，访问权一旦授出，我们可能就收不回来了，因为这个进程可能在别处藏了这个能力的副本[6]。再者，这个进程也许只要通过 IPC 把能力的副本传给另一个进程，就能把访问权授予那个进程。

用能力做访问控制时，我们通常这样处理这些问题：绝不让进程把（比喻意义上的）手伸到任何能力上。能力由操作系统控制和维护，存放在操作系统受保护的内存空间中的某个地方。进程可以对能力执行各种操作，但只能在操作系统的仲裁下进行。例如，如果进程 A 想用能力授予进程 B 对文件 /tmp/foo 的读写权，A 不能直接把相应的位模式发给 B。A 必须发起一个系统调用，请求操作系统把相应的能力交给 B。这样，操作系统就有机会判断它的安全策略是否允许 B 访问 /tmp/foo；如果不允许，就拒绝这次能力转移。

所以，如果要依靠能力做访问控制，操作系统就得为每个进程维护一张受它保护的能力列表。这很简单，因为操作系统本来就有一个每进程一份、受保护的数据结构：PCB。在进程的 PCB 里放一个指向能力列表（存放在内核内存中）的指针，就万事俱备了。现在，当进程试图以读写方式打开 /tmp/foo 时，调用会陷入操作系统，操作系统查看这个进程的能力列表，看上面有没有针对这个操作的相关能力，然后照此处理。

在通用系统中，要在线保存一张列表，把某个主体获准访问的所有东西都列进去，开销会很高。如果我们用能力来做基于文件的访问控制，一个用户可能就有成千上万个能力，他能以任何方式访问的每个文件都对应一个。一般来说，使用能力的系统会把能力持久地存放在某个安全的地方，需要时再导入。所以，附在进程上的能力列表不一定很长；但有一个问题要决定：用户手头可以自由支配的能力数量庞大，该把其中哪些交给他运行的每个进程？

还有另一种选择。能力不一定要存放在操作系统中，而是可以用密码学（cryptography）来保护。如果能力比较长，而且是用强密码学生成的，那它实际上就无法被猜出来，可以留

[6] 收回访问权的这种做法通常称为**撤销**（revocation）。对 ACL 来说，撤销很容易，只要找到 ACL 改掉就行。对能力来说，撤销是难是易，取决于具体的实现。

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

在用户手里。密码学保护的能力最适合用在分布式系统中，所以我们会在分布式系统安全一章中讨论它们。

和访问控制列表（access control list）一样，能力也有优点和缺点。用能力时，很容易确定某个主体能访问哪些系统资源，只要把这个主体的能力列表看一遍就行了。撤销访问权只需把能力从列表中删掉；如果操作系统独占这个能力，这很容易做到（如果不是独占，就难多了）。如果能力就在内存里随手可用，检查它的开销可以非常低，尤其是能力本身就可以包含一个指针，指向它所保护的资源对应的数据或软件。也许，系统对能力的核心实现，就只是持有这样一个指针而已。

另一方面，要确定能访问某个资源的全部主体，开销就大了。任何主体都可能持有这个资源的能力，所以你必须检查所有主体的能力列表才能知道。而且，能让能力列表保持简短、易于管理的简单方法，也不像 Unix 那种简短 ACL 的方法那样成熟。此外，系统创建、存储和取出能力的方式必须能杜绝伪造，这可能很有挑战性。

能力有一个很妙的地方：它提供了一种创建特权受限的进程的好办法。用访问控制列表（access control list）时，进程会继承父进程（parent process）的身份，因而也继承了那个主体的全部特权。很难只把父进程特权的一个子集交给这个进程。要么得创建一个只有这些受限特权的新主体，修改一堆访问控制列表（access control list），再把新进程的身份设成这个新主体；要么得对访问控制模型做某种扩展，而这种扩展的行为和访问控制列表（access control list）通常的行为不太一样。用能力就很容易。如果父进程有 X、Y、Z 三个能力，但只想让子进程（child process）拥有 X 和 Y，那么创建子进程时，父进程就只转给它 X 和 Y，不转 Z。

实际上，出于种种原因，用户能看到的访问控制机制往往使用访问控制列表（access control list），而不是能力。不过在底层，操作系统大量使用能力。例如，在一个典型的 Linux 系统中，我们一直在讨论的那个 `open()` 调用是用 ACL 做访问控制的。但假设 Linux 的 `open()` 成功了，那么只要进程一直打开着这个文件，后续的读和写就不会再检查 ACL。Linux 会创建一个数据结构，它实际上就相当于一个能力，表明这个进程对该文件有读写特权。这个结构附在进程的 PCB 上。每次读写时，操作系统只需查看这个数据结构，就能判断是否允许读写，而不必去找文件的访问控制列表（access control list）。如果文件被关闭，这个类似能力的结构就会从 PCB 中删除，进程要再访问这个文件，就得重新调用一次 `open()`，而那又要回到 ACL 去检查。类似的技术也可以用来控制对硬件设备和 IPC 通道的访问，尤其是因为类 UNIX 系统把这些资源都当作文件来对待。ACL 和能力结合使用，让系统避开了各自机制的一些问题。每次操作都检查访问控制列表（access control list）的开销省下来了，因为这种形式的能力很容易检查，它不过是操作系统数据结构中某个指针在或不在而已。为所有可访问客体（object）管理能力的开销也避免了，因为只有在 ACL 检查成功之后才会建立能力。如果进程从不访问某个客体（object），就永远不会检查它的 ACL，也不需要为它建立能力。由于任何一个进程通常只会打开它获准打开的全部文件中极小的一部分，可扩展性（scalability）问题一般不会出现。

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

55.5 强制访问控制与自主访问控制

计算机资源的访问控制应该是什么样，该由谁来决定？对大多数人来说，答案似乎显而易见：谁拥有这个资源，就由谁决定。对用户的文件来说，应当由这个用户决定访问控制设置。对系统资源来说，应当由系统管理员，或者也许由这台计算机的主人来决定。然而，对某些系统和某些安全策略来说，这个答案并不对。特别是，最关心信息安全的那些人，有时希望控制得比这更严。

军方是最明显的例子。我们都听说过绝密（Top Secret）信息，大概也都明白，即使你获准查看绝密信息，你也不应该让别人看到它。即使这些信息就在你自己创建的文件里，也是如此，比如你写的一份报告引用了另一份绝密文件里的统计数据或原文。在这些情况下，由创建者控制访问权限这个简单的答案就不对了。必须由机构中全面负责信息安全的人来作这些决定。这意味着，这个主体有权给其他用户创建和拥有的信息设置访问控制，而且这些用户推翻不了他的决定。比较常见的情况称为**自主访问控制**（discretionary access control）：给几乎所有人还是几乎不给任何人访问某个资源的权限，由拥有它的用户自行决定。限制更严的情况称为**强制访问控制**（mandatory access control）：在这类系统中，访问控制决定中至少有一部分是由某个权威机构强制规定的，它可以推翻信息所有者的意愿。选择自主访问控制（discretionary access control）还是强制访问控制（mandatory access control），与使用 ACL 还是能力是正交的，而且通常也独立于访问控制机制的其他方面，比如访问信息如何存储和处理。强制访问控制（mandatory access control）系统也可以包含一些自主的成分，允许对强制控制作进一步限制（但不能放松）。

很多人永远不会用到运行强制访问控制（mandatory access control）的系统，所以我们不打算深入讨论它们如何工作，只指出一点：显然，实施这类控制离不开操作系统的参与。如果你哪天需要在一个看重强制访问控制（mandatory access control）的环境中工作，你一定会听说它的。到那时你应当再多了解一些，因为既然有人在乎到要使用强制访问控制（mandatory access control）机制，他们也就在乎到会惩罚不守规矩的用户。Loscocco [L01] 介绍了一个加入了强制访问控制（mandatory access control）的特殊版本的 Linux。如果你想进一步了解这类系统的特点，这是一篇很好的入门论文。

55.6 访问控制机制的实际问题

大多数系统都向用户提供一种访问控制列表（access control list）机制，或简单，或功能更强；而且其中大多数使用自主访问控制（discretionary access control）。然而，一台现代计算机很容易就有几十万甚至几百万个文件，让人类用户逐个设置访问控制权限是不可行的。一般来说，系统允许每个用户设定一个默认访问权限，用于他创建的每个文件。如果用 Linux 的 `open()` 调用创建文件，可以指定最初赋给这个文件的访问权限。在 Unix/Linux 系统中，新建文件的访问权限还可以用 **`umask()`** 调用进一步控制，它作用于调用它的那个进程此后创建的所有新文件。

如果愿意，所有者可以修改这个初始 ACL，但经验表明，用户很少这么做。这种倾向说明了选好默认值有多重要。和操作系统中的许多其他地方一样，这里的设置理论上可以修改或调整，实际上却几乎所有人几乎总是原封不动地用着。

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

补充：Android 的访问控制模型

Android 系统是当今移动计算设备（尤其是智能手机）上领先的软件平台之一。这些设备带来的访问控制难题，和传统的服务器计算机，甚至个人台式机或笔记本电脑都不一样。它们的功能建立在许多相对较小的独立应用程序之上，这些程序通常称为 app，下载、安装并运行在只属于一个用户的设备上。所以，不存在要在一台机器上保护多个用户互不侵犯的问题。如果使用标准的访问控制模型，这些 app 就会以那个用户的身份运行。可是 app 由许多不同的实体开发，其中一些可能是恶意的。此外，大多数 app 并没有正当理由使用设备上的大多数资源。如果给它们的特权过多，恶意 app 就能访问机主的通讯录、拨打电话、在网上买东西，干出许多别的坏事。最小特权（least privilege）原则意味着我们不应该把机主的全部特权都交给 app，但它们要做点有意思的事，又必须有一些特权。

Android 运行在某个版本的 Linux 之上，它对应用程序访问的限制，部分是通过给每个安装的 app 生成一个新的用户 ID 来实现的。app 以这个 ID 运行，就可以据此控制它的访问。不过，Android 中间件还提供了额外的访问控制手段。应用程序开发者要声明自己的 app 需要哪些访问。用户考虑在设备上安装某个 app 时，会看到它需要哪些权限。用户可以授予 app 这些权限，也可以不安装这个 app，或者限制它的权限，不过限制权限也可能影响 app 的用途。此外，开发者还要规定其他 app 能以哪些方式与这个新 app 通信。用来编码这些访问信息的数据结构称为 **permission label**。一个 app 的 permission label（既包括它能访问什么，也包括它向其他 app 提供什么）在 app 设计时设定，并在 app 安装到某台 Android 设备上的那一刻编码进这个系统。

所以，permission label 很像能力：app 持有某个 label，就可以做某件事；缺少这个 label，就做不了那件事。一个 app 的 permission label 集合在安装时静态设定。用户之后可以修改这些权限，不过限制权限可能会损害 app 的功能。permission label 是强制访问控制（mandatory access control）的一种形式。Enck 等人 [E+09] 详细讨论了 Android 的安全模型。

Android 的安全方法很有意思，但并不完美。尤其是，用户并不总能意识到授予某个应用程序访问某样东西意味着什么；当面临要么授予访问权、要么没法正常使用这个 app 的选择时，他们往往会授予。如果这个 app 是恶意的，这种做法就可能出问题。

不过，虽然很多人永远不会碰自己资源上的访问控制，但对一批重要的用户和系统来说，这些控制对实现他们的安全目标至关重要。即使你大多依赖默认值，许多软件安装包在给它们创建的可执行文件和配置文件设置访问控制时，也会相当谨慎。一般来说，摆弄系统中的访问控制时应当小心。如果你不知道自己在做什么，就可能泄露敏感信息，或者让攻击者能篡改关键的系统设置。如果你收紧了现有的访问控制，可能会突然让后台运行的一堆守护进程（daemon）停止工作。

许多大型机构在试图用标准的访问控制方法实施安全策略时，发现了一个实际问题：在机构中担任不同角色的人需要不同的特权。例如，在一家医院里，所有医生可能都有一组药剂师没有的特权，而药剂师又有一些医生没有的特权。按照这样的角色来组织访问控制，再

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

把具体用户分配到他们获准担任的角色上，许多安全策略实现起来就会容易一些。如果某些用户可以根据当前执行的任务切换角色，这种方法就尤其有价值，因为这样就不必临时设置或修改个人的访问权限，只需把他们从一个角色切换到另一个角色。通常，他们只在担任某个角色期间才拥有这个角色的权限。一旦退出某个角色（也许是为了进入另一个特权不同的角色），他们就失去了所退出角色的特权。

这一观察催生了**基于角色的访问控制**（role-based access control，**RBAC**）。它的核心思想早已存在，后来才由 Ferraiolo 和 Kuhn 在一篇研究论文中更正式地阐述出来 [FK92]。现在，RBAC 在许多机构中都很常用，尤其是大型机构。大型机构面临的管理难题比小机构严重，所以像 RBAC 这样能一次操作处理一批用户的方法，可以大大减轻管理工作。例如，如果一家公司决定，所有程序员都应当获得访问一个新开发的库的权限，而会计不应当，那么用 RBAC 只需一次操作，把所需的特权赋给 *Programmer*（程序员）角色，就能达到这个效果。如果某个程序员被提拔到一个不需要访问这个库的管理岗位，公司只需把 *Programmer* 角色从这位经理可以担任的角色集合中去掉。

这样的限制并不一定意味着你怀疑自己的会计不诚实，会把公司的秘密库代码卖给竞争对手[7]。记住最小特权（least privilege）原则：你给别人访问某样东西的权限时，依靠的不仅是他们的诚实，还有他们的谨慎。如果会计根本无法访问这个库，那么无论是他们的恶意还是疏忽，都不会让会计的特权导致你的库代码泄露。最小特权（least privilege）不仅在理论上是个好主意，在现实世界中构建安全系统时也至关重要。

RBAC 听起来有点像在访问控制列表（access control list）中使用组，两者确实有些相似，但 RBAC 系统比单纯的组访问权限强大得多：RBAC 系统允许某个用户担任多个互不相交的角色。也许我们那位程序员被提拔到了管理岗位，但仍然需要访问那个库，比如需要测试另一位团队成员的代码时。RBAC 系统会让这位程序员在经理和程序员两个角色之间切换：暂时放下与经理角色相关的权利，获得与程序员角色相关的权利。当经理测试别人的新代码时，他有权访问那个库，但没有权限查看团队成员的绩效评估。这样，如果某个狡猾的程序员往库里偷偷塞了恶意代码（比如试图读取其他团队成员的绩效评估，或者打探他们的工资），运行这段代码的经理也不会无意中泄露这些信息；在合适的时候使用合适的角色，就能防止这种事情发生。

这类系统通常要求重新进行一次身份认证，才能担任某个 RBAC 角色；而且担任角色 A 通常意味着要放弃先前角色（比如角色 B）的相关特权。经理切换到代码测试角色时，就会暂时放弃查看绩效评估的特权。测试完成后，经理再切换回可以查看评估的角色，同时失去访问那个库的特权。RBAC 系统还可以提供比单纯读写文件更细的粒度。某个特定角色（比如 *Salesperson*，销售员）也许可以往一个文件中添加某种产品的购买记录，却不能往同一个文件中添加同一种产品的补货记录，因为销售员不负责补货。这种程度的控制有时称为**类型强制**（type enforcement）。它借助通常所说的客体（object）的**安全上下文**（security context），把详细的访问规则与特定的客体关联起来。具体怎样实现，会影响性能、安全上下文（security context）信息的存储，以及身份认证。

在 Linux 和类似的操作系统下，可以用 ACL 和组搭建一个非常简单的 RBAC 系统。这些系统的访问控制机制有一个特性，称为**权限提升**（privilege escalation）。权限提升允许

[7] 不诚实的会计一般最好避开，所以你多半已经尽力雇了诚实的。除非你是 Bernie Madoff [W20]，也许……

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

提示：权限提升是危险的

我们刚刚讲了怎样用权限提升临时改变某个用户能做的事，以及这如何给我们提供了新的安全手段。但权限提升也有危险的一面。攻击者攻破你的系统时，攻陷的往往是一个只有很有限特权的程序。也许它本来只能处理几个简单的信息文件，把文件内容提供给远程用户，或许再对这些文件运行一些标准工具。它甚至可能对自己的文件都没有写权限。你可能会觉得这种攻陷对系统没造成什么危害，因为攻击者利用这种访问权做不了多少事。

这正是权限提升的危险所在。在系统上取得了任何一点立足之地的攻击者，接下来就会四处寻找提升权限的途径。即使是一个特权相当少的应用程序，也能做许多外人无法直接做的事，所以攻击者会在被攻陷的应用程序能访问的代码或配置中寻找漏洞。成功攻陷一个系统之后，攻击者通常头一件事就是这样尝试提升权限。

在许多系统中，有一个特殊的用户，通常称为**超级用户**（superuser）或 **root** 用户。这个用户的特权比系统中其他任何用户都多得多，因为它的用途就是让人能对系统做最关键、影响最深远的管理变更。在你的系统上站稳了脚跟的攻击者，首要目标就是利用权限提升成为 root 用户。能做到这一点的攻击者，实际上就完全控制了你的系统。这样的攻击者可以查看任何文件、修改任何程序、更改任何配置，甚至可能安装另一个操作系统。这种危险应当让你明白：对于任何可能一路提升到超级用户特权的途径，都必须极其小心。

谨慎地扩展特权，通常的做法是允许某个特定程序以超出调用者本人特权的一组特权运行。在 Unix 和 Linux 系统中，这个特性叫作 **setuid**，它允许一个程序以另一个用户的特权运行，这个用户通常拥有运行该程序的用户平常没有的特权。不过，这些特权只在程序运行期间授予，程序一退出就失去了。一个精心编写的 setuid 程序只会用这些特权执行有限的一组操作，以确保特权不会被滥用[8]。我们可以为每个角色定义一个虚拟用户，并把所需的特权和这个用户关联起来，从而搭建一个简单的 RBAC 系统。需要这些特权的程序，可以设成以这个用户的身份 setuid 运行。

我们在身份认证一章中遇到过的 Linux `sudo` 命令就提供了这类功能，它允许某些指定的用户以另一个身份运行某些程序。例如：

```
sudo -u Programmer install newprogram
```

这条命令会以用户 Programmer 的身份，而不是以运行这条命令的用户的身份，运行这个 install 命令；前提是该用户在系统维护的一张名单上，名单上的用户获准担任 Programmer 这个身份。要安全地使用这种方法，就得仔细配置那些控制“谁可以以哪个身份执行哪些程序”的系统文件。和其他 RBAC 系统一样，`sudo` 命令通常要求重新进行一次身份认证。

为了更高级的用途，RBAC 系统通常支持比 setuid 和 `sudo` 操作更细的粒度，对角色分配的跟踪也更细致。这样的 RBAC 系统可能是操作系统的一部分，也可能是系统的某种

[8]遗憾的是，并非所有以 setuid 方式运行的程序都写得很仔细，这些年来已经导致了许多安全问题。唉，也许所有安全特性都是如此?

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

附加组件，或者是一个编程环境。使用 RBAC 时，往往也会运行一定程度的强制访问控制（mandatory access control）。如果不这样，在上面 `sudo` 的例子中，以 Programmer 身份运行的用户就可以运行一条命令修改文件的访问权限，让非程序员也能使用那个 install 命令。有了强制访问控制（mandatory access control），用户可以担任 Programmer 角色来完成安装，却不能利用这个角色让销售员或会计也能执行安装。

55.7 小结

实施大多数安全策略，都需要控制哪些用户可以以哪些方式访问哪些资源。操作系统内置的访问控制机制提供了所需的功能。好的访问控制机制会借助精心设计和实现的 reference monitor，对与安全相关的访问做到完全仲裁（complete mediation），或者接近完全仲裁。

访问控制列表（access control list）和能力是大多数访问控制系统使用的两种基本机制。访问控制列表（access control list）精确地规定了哪些主体可以以哪些方式访问哪些客体（object），在不在相应的列表上决定了是否批准访问。能力更像是锁的钥匙，持有正确的能力，就足以证明应当允许访问某个资源。用户可见的访问控制更常通过某种形式的访问控制列表（access control list）来实现，而能力常常内置在操作系统中用户看不到的那一层。这两种访问控制机制本身并无优劣之分。和系统设计中的许多选项一样，它们各有特点，在有些场合和用途中很合适，在另一些场合和用途中则不合适。你需要明白在什么情况下该选哪一种。

访问控制机制可以是自主的，也可以是强制的，有些系统两者兼有。类型强制（type enforcement）和基于角色的访问控制（role-based access control）等增强手段，可以让你更容易实现所需的安全策略。

即使访问控制机制完全正确、极其高效，它能做的也不过是实施交给它的安全策略。因访问控制机制本身有缺陷而导致的安全失败很少见；因为这些机制所实施的策略设计得很差而导致的安全失败，可一点也不少见。

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

参考文献

[C+63] "The Compatible Time Sharing System: A Programmer's Guide" by F. J. Corbato, M. M. Daggett, R. C. Daley, R. J. Creasy, J. D. Hellwig, R. H. Orenstein, and L. K. Korn. M.I.T. Press, 1963. *The programmer's guide for the early and influential CTSS time sharing system. Referenced here because it used an early version of an access control list approach to protecting data stored on disk.*

[DV64] "Programming Semantics for Multiprogrammed Computations" by Jack B. Dennis and Earl. C. van Horn. Communications of the ACM, Vol. 9, No. 3, March 1966. *The earliest discussion of the use of capabilities to perform access control in a computer. Though the authors themselves point to the "program reference table" used in the Burroughs B5000 system as an inspiration for this notion.*

[E+09] "Understanding Android Security" by William Enck, Machigar Ongtang, and Patrick McDaniel. IEEE Security and Privacy, Vol. 7, No. 1, January/February 1999. *An interesting approach to providing access control in a particular and important kind of machine. The approach has not been uniformly successful, but it is worth understanding in more detail than we discuss in this chapter.*

[FK92] "Role-Based Access Controls" by David Ferraiolo and D. Richard Kuhn. 15th National Computer Security Conference, October 1992. *The concepts behind RBAC were floating around since at least the 70s, but this paper is commonly regarded as the first discussion of RBAC as a formal concept with particular properties.*

[L84] "Capability-Based Computer Systems" by Henry Levy. Digital Press, 1984. *A full book on the use of capabilities in computer systems, as of 1984. It includes coverage of both hardware using capabilities and operating systems, like Hydra, that used them.*

[L01] "Integrating Flexible Support for Security Policies Into the Linux Operating System" by Peter Loscocco. Proceedings of the FREENIX Track at the USENIX Annual Technical Conference 2001. *The NSA built this version of Linux that incorporates mandatory access control and other security features into Linux. A good place to dive into the world of mandatory access control, if either necessity or interest motivates you to do so.*

[S74] "Protection and Control of Information Sharing in Multics" by Jerome Saltzer. Communications of the ACM, Vol. 17, No. 7, July 1974. *Sometimes it seems that every system idea not introduced in CTSS was added in Multics. In this case, it's the general use of groups in access control lists.*

[T84] "Reflections on Trusting Trust" by Ken Thompson. Communications of the ACM, Vol. 27, No. 8, August 1984. *Ken Thompson's Turing Award lecture, in which he pointed out how sly systems developers can slip in backdoors without anyone being aware of it. People have wondered ever since if he actually did what he talked about…*

[W20] "Bernie Madoff" by Wikipedia. https://en.wikipedia.org/wiki/Bernie_Madoff. *Bernie Madoff (painfully, pronounced "made off", as in "made off with your money") built a sophisticated Ponzi scheme, a fraud of unimaginable proportions (nearly 100 billion dollars). He is, as Wikipedia says, an "American charlatan". As relevant here, he probably hired dishonest accountants, or was one himself.*

[W+74] "Hydra: The Kernel of a Multiprocessor Operating System" by W. Wulf, E. Cohen, W. Corwin, A. Jones, R. Levin, C. Pearson, and F. Pollack. Communications of the ACM, Vol. 17, No. 6, June 1974. *A paper on a well-known operating system that made extensive and sophisticated use of capabilities to handle access control.*

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)